2 DI asılılığı kənardan verməkdir

Sifariş tamamlananda qəbz hazırlayan kiçik proqram quracağıq. Əvvəl işləyən DI-sız kodu, sonra eyni işi DI ilə görən kodu oxuyacağıq. Nəticə dəyişməyəcək; obyektlərin harada yaradıldığı və necə birləşdirildiyi dəyişəcək.

DI - Dependency Injection bir obyektin ehtiyac duyduğu başqa obyektin ona kənardan verilməsidir. Dependency asılılıq, injection isə həmin asılılığın ötürülməsi deməkdir.

Bu dərsdə anlayışları aşağıdakı sıra ilə açırıq:

- Əvvəl DI-sız və DI ilə kod, sonra nəzəri fərq.
- Interface ilə konkret sinfə bağlılığın azaldılması.
- Container-in gördüyü iş.
- Sorğu və scope: bir işin başlanğıc və bitiş sərhədi.
- Resolve: container-dən xidmət əldə etmək.
- Transient, scoped və singleton: yeni və ya eyni obyektin verilməsi.
- İşlədilə bilən Console və veb nümunələri.
- Injection üsulları, lifetime səhvləri və resursların təmizlənməsi.

İlk oxunuşda real veb nümunəsinin nəticələrinə qədər ardıcıl get. Sonrakı səhifələr artıq öyrəndiyin məntiqi genişləndirir. Əlavə terminlər əsas nümunəyə ehtiyac yarandıqda təqdim edilir.

Kodlarda qəbz hazırlamaq sadəcə mətn qaytarmaqdır. Printer cihazına qoşulmur, e-poçt göndərilmir, PDF yaradılmır. Beləliklə, diqqət yalnız obyektlər arasındakı əlaqədə qalır.

Bu dərsin əsas sualı: lazım olan obyekti kim yaradır, kim verir və kim istifadə edir?

DI-sız proqram: printer-i sifariş sinfi özü yaradır

Bu, Console layihəsinin tam `Program.cs` nümunəsidir. Sonrakı alternativ nümunələrdə həmin fayl əvəz edilir; eyni adlı sinifləri bir fayla təkrar əlavə etmirik.

C# - Program.cs, DI-sız variant

```
var orders = new OrderService();
Console.WriteLine(orders.Complete());

public class Printer
{
    public string Print()
    {
        return "Qebz hazirdir";
    }
}

public class OrderService
{
    private Printer _printer = new Printer();

    public string Complete()
    {
        return _printer.Print();
    }
}
```

Ekrandakı nəticə

```
Qebz hazirdir
```

`OrderService` yarananda onun `_printer` sahəsi üçün də `new Printer()` işləyir. Sifariş obyekti öz asılılığını özü yaradır. Bu işləyən koddur; məsələ onun dəyişikliklərə necə uyğunlaşacağıdır.

DI-sız kodda hər sətrin mənası

Sinif - class obyektin quruluşunu və edə bildiyi əməliyyatları müəyyən edir. `Printer` və `OrderService` sinif adlarıdır. **Obyekt - instance** həmin sinifdən yaradılan konkret nüsxədir.

`new OrderService()` sifariş obyektini yaradır. `var orders` həmin obyektə istinadı saxlayan dəyişəndir. `var` dəyişənin tipinin sağdakı ifadədən müəyyən edilməsinə imkan verir.

Yaradılma və istifadə ardıcıllığı

```
1. new OrderService() isleyir.
2. Onun sahesi ucun new Printer() isleyir.
3. orders.Complete() cagirilir.
4. Complete daxilinde _printer.Print() cagirilir.
5. Metn neticesi ekrana yazilir.
```

Sahə - field obyektin daxilində saxlanan məlumatdır. `_printer` belə sahədir. _ işarəsi sadəcə adlandırma seçimidir. `private` sahənin yalnız həmin sinfin daxilində əlçatan olduğunu bildirir.

Metod çağırılan əməliyyatdır. `Complete` və `Print` metod adlarıdır. `string` mətn tipidir. `return` nəticəni çağıran koda verir. `Console.WriteLine` onu ekrana yazır. `public` hissənin kənardan əlçatan olduğunu göstərir.

`OrderService`-in işləməsi `Printer` obyektinə bağlıdır. Buna görə Printer onun **asılılığıdır**. Hazırkı kod həm sifarişi tamamlayır, həm də hansı printer növünün yaradılacağına qərar verir.

Sabah başqa qəbz hazırlayıcısı istəsən, yaradılma seçimi OrderService-in daxilində olduğu üçün həmin sinfə toxunmalısan. **Coupling** bu cür bağlılıqdır: bir hissənin dəyişməsi digər hissəyə də dəyişiklik yayır.

DI ilə proqram: printer-i kənarda yaradıb veririk

İndi eyni nəticəni alaq, amma Printer-in yaradılmasını OrderService-dən çıxaraq. Bu da tam alternativ `Program.cs` nümunəsidir.

C# - Program.cs, constructor injection

```
var printer = new Printer();
var orders = new OrderService(printer);
Console.WriteLine(orders.Complete());

public class Printer
{
    public string Print()
    {
        return "Qebz hazirdir";
    }
}

public class OrderService
{
    private Printer _printer;

    public OrderService(Printer printer)
    {
        _printer = printer;
    }

    public string Complete()
    {
        return _printer.Print();
    }
}
```

Nəticə yenə **Qebz hazirdir** olur. Printer-i indi proqramın başlanğıcındakı kod yaradır və OrderService-ə verir. **Bu artıq DI-dır; ayrıca container istifadə etməmişik.**

Constructor injection obyekt yaratmaqdan necə ayrılır

Konstruktor - constructor obyekt yaradılarkən onun ilkin vəziyyətini quran xüsusi hissədir. Siniflə eyni adda yazılır. Əvvəlki nümunədə `OrderService(Printer printer)` konstruktor idi.

C# - əvvəlki proqramın iki sətri

```
var printer = new Printer();
var orders = new OrderService(printer);
```

Birinci `new` Printer obyektini yaradır. İkinci `new` OrderService obyektini yaradır və onun konstruktoruna mövcud Printer-i verir. **Injection ötürmə addımıdır.** Konstruktorun parametrində Printer yazılması öz-özünə Printer yaratmır.

`printer` konstruktora gələn **parametr**, `_printer` isə onu saxlayan sahədir. Parametr çağırış zamanı verilən dəyəri qəbul edir. `_printer = printer` həmin obyektə istinadı sahədə saxlayır.

İstinad - reference obyektə çatmağa imkan verən dəyərdir. Sinif obyektini belə ötürəndə obyektin surəti yaradılmır:

Eyni obyektə iki yoldan çatırıq

```
Program-daki printer  ----+
                          +----> Printer obyekti
orders daxilinde _printer-+
```

Məsələn, `var second = printer;` yazsaq da yeni Printer yaranmır. `second` və `printer` eyni obyektə istinad edir. **ReferenceEquals** iki istinadın eyni obyektə aid olub-olmadığını yoxlayır.

Konstruktora `null`, yəni obyekt olmayan boş dəyər də ötürmək mümkündür. Məcburi asılılıq üçün constructor-da bunu yoxlamaq olar. Constructor parametrinin olması təkbaşına bütün yanlış istifadələri aradan qaldırmır.

Interface hansı qəbz növünün dəyişə biləcəyini göstərir

Əllə DI etdik, amma OrderService hələ konkret Printer sinfini tanıyır. **Interface** tələb olunan əməliyyatları müəyyən edən müqavilədir. Sifarişə konkret cihaz və ya fayl təfərrüatı deyil, qəbz hazırlamaq əməliyyatı lazımdır.

C# - ayrıca tip tərifləri

```
public interface IPrinter
{
    string Print();
}

public class Printer : IPrinter
{
    public string Print()
    {
        return "Adi qebz";
    }
}

public class PdfPrinter : IPrinter
{
    public string Print()
    {
        return "PDF qebzi";
    }
}
```

`IPrinter` müqaviləsi mətn qaytaran Print əməliyyatı tələb edir. `: IPrinter` həmin müqavilənin yerinə yetirildiyini bildirir. Konkret həllə **implementation - reallaşdırma** deyilir. Printer və PdfPrinter iki reallaşdırmadır.

Interface-in özünü `new IPrinter()` ilə yaratmırıq. Müqaviləni yerinə yetirən konkret sinfin obyektini yaradırıq. Burada PdfPrinter də sadəcə izah üçün mətn qaytarır; real PDF hazırlamır.

Eyni sifariş koduna iki fərqli printer verək

Əvvəlki səhifənin IPrinter, Printer və PdfPrinter tərifləri layihədə qalır. OrderService isə artıq müqaviləni qəbul edir:

C# - OrderService.cs

```
public class OrderService
{
    private IPrinter _printer;

    public OrderService(IPrinter printer)
    {
        _printer = printer;
    }

    public string Complete()
    {
        return _printer.Print();
    }
}
```

C# - Program.cs

```
var ordinary = new OrderService(new Printer());
var pdf = new OrderService(new PdfPrinter());

Console.WriteLine(ordinary.Complete());
Console.WriteLine(pdf.Complete());
```

Nəticələr ardıcıl **Adi qebz** və **PDF qebzi** olur. OrderService-in daxilini dəyişmədik; ona verilən konkret həlli dəyişdik.

Loose coupling - zəif bağlılıq bir hissəni dəyişəndə dəyişikliklərin digər hissələrə daha az yayılmasıdır. Interface konkret printer-in seçimini sifarişin əsas məntiqindən ayırmağa kömək etdi. DI interface olmadan da var idi; interface əlavə dəyişiklik sərbəstliyi verdi.

Nəzəri fərqi testdə görək

Test kodun gözlənilən davranışını yoxlamaqdır. **Fake** isə yoxlama üçün yazılmış sadə əvəzedicidir. Həqiqi printer əvəzinə əvvəlki müqaviləyə uyğun əvəzedici verə bilərik:

C# - FakePrinter.cs

```
public class FakePrinter : IPrinter
{
    public string Print()
    {
        return "Test qebzi";
    }
}
```

C# - Program.cs

```
var orders = new OrderService(new FakePrinter());
Console.WriteLine(orders.Complete());
```

Nəticə **Test qebzi** olur. Printer cihazına və əlavə test kitabxanasına ehtiyac olmadı. Əvəzedicini özümüz verə bildik.

Məsələ	DI-sız ilk nümunə	Interface və DI ilə
Obyekti kim yaradır	OrderService	Kənardakı kod
Konkret növü kim seçir	OrderService	Kənardakı kod
OrderService nəyi tanıyır	Printer sinfini	IPrinter müqaviləsini
Test əvəzedicisi	Daxili yaradılmaya müdaxilə tələb edir	Konstruktorla verilə bilir

new yoxa çıxmadı. Obyekt yaratmaq yenə lazımdır. Konkret asılılığı yaratmaq qərarını onu istifadə edən biznes sinfindən kənara çıxardıq. Biznes burada sifarişi tamamlamaq kimi tətbiqin əsas iş qaydasını bildirir.

Container əllə gördüyümüz birləşdirməni təşkil edir

İki obyekti əllə birləşdirmək asandır. Amma böyük proqramda hər sinfin öz asılılıqları ola bilər. Bu əlaqələri bir yerə toplamaq üçün **DI container** istifadə edilir.

Container hansı xidmətin necə təmin olunacağını, hansı hallarda mövcud obyektin paylaşılacağını və idarə etdiyi resursların nə vaxt təmizlənəcəyini müəyyən edən qeydiyyatlarla işləyir.

Xidmət - service burada başqa kodun istifadə etdiyi köməkçi obyektdir. Printer və OrderService xidmət ola bilər. Bu söz ayrıca server və ya şəbəkədə işləyən mikroservis mənasına gəlmir.

OrderService tələb olunanda

```
1. Onun konstruktorundaki IPrinter ehtiyaci gorulur.
2. Qeydiyyatdan IPrinter-in konkret helli tapilir.
3. Helle uygun Printer obyekti elde edilir.
4. Printer konstruktorla OrderService-e verilir.
5. Hazir OrderService istifade edecek koda qaytarilir.
```

Asılılığın özünün də constructor ehtiyacları varsa onlar da bu qayda ilə təmin olunur. Bu əlaqələrə **object graph - obyektlər əlaqəsi** deyilir. Bu nümunədə əlaqə sadədir: OrderService Printer-dən istifadə edir.

DI ötürmə üsuludur; container bu ötürmələri təşkil edən vasitədir. Container olmadan yazdığımız constructor injection da tam DI idi. Container-dən istifadədə isə sinif ehtiyacını bildirir, yaratma və paylaşma qaydaları kənarda saxlanılır.

Scope-dan əvvəl HTTP sorğusunu anlayaq

Client - müştəri proqramı işi istəyən tərəfdir: brauzer və ya telefondakı tətbiq. **Server** bu istəyi qəbul edib cavab verən tərəfdir.

Request - sorğu client-in serverə göndərdiyi iş istəyidir. **Response - cavab** serverin qaytardığı nəticədir. HTTP bu sorğu və cavabların göndərilmə qaydalarını müəyyən edir.

Bir HTTP sorğusu

```
Telefon: "Sifarislerimi goster."
    |
    v
Server: istifadecini mueyyen edir.
        bazadan sifarisleri oxuyur.
        cavabi hazirlayir.
    |
    v
Telefon: neticeni ekranda gosterir.
```

ASP.NET Core C# ilə veb server tətbiqi qurmaq üçün istifadə olunan çərçivədir. O, sorğunun uyğun koda çatdırılması və cavabın göndərilməsi kimi işləri təşkil edir.

Eyni istifadəçi əvvəl menyunu, sonra sifarişlərini istəyə bilər. Bunlar iki ayrı HTTP sorğusudur. Sorğu istifadəçinin bütün sistemdə qalma müddəti deyil. Eyni vaxtda müxtəlif istifadəçilərin sorğuları da işlənə bilər.

İndi bir sorğunun başlanğıcından sonuna qədər istifadə olunan xidmətlərin idarə edilməsinə sərhəd qoymaq ehtiyacı yaranır. Növbəti səhifədə bu sərhədə scope deyəcəyik.

Scope bir iş üçün açılan idarəetmə sərhədidir

Scope müəyyən iş ərzində xidmətlərin paylaşılmasına və təmizlənməsinə sərhəd qoyan sahədir. Adi ASP.NET Core HTTP tətbiqində framework hər yeni sorğu üçün ayrıca scope açır.

Birinci sorğu

```
Scope A acilir.
    Istifadeci mueyyen edilir.
    Sifarisler oxunur.
    Cavab hazirlanir.
Scope A baglanir.

Ikinci sorğu
Scope B acilir.
    Yeni sorgunun isi gorulur.
Scope B baglanir.
```

Hər HTTP sorğusuna ayrıca scope deyəndə adi sorğu emalı nəzərdə tutulur. Proqramçı ayrıca iş sərhədi üçün əlavə scope da aça bilər. Beləliklə, scope anlayışı yalnız HTTP-yə aid deyil.

Scope xidmətləri idarə etmək üçün sərhəddir. Hansı xidmətin scope daxilində eyni obyekt olacağını isə ayrıca paylaşılma qaydası müəyyən edir. Ona görə **yeni scope açılması bütün xidmət obyektlərinin mütləq yenidən yaradılması demək deyil**.

Scope bağlananda onun idarə etdiyi təmizlənmə işləri görülür. Bu, bütün obyektlərin həmin saniyə yaddaşdan silinməsi mənasına gəlmir. Resursun buraxılması ilə yaddaşın geri alınmasını dərsin sonrakı hissəsində ayıracağıq.

HTTP scope-unu EF Core deyil ASP.NET Core açır

EF Core C# obyektləri ilə verilənlər bazası arasında işləməyi asanlaşdıran kitabxanadır. **DbContext** baza sorğuları və obyekt dəyişiklikləri ilə işləmək üçün onun əsas obyektidir.

Adi veb tətbiqində HTTP sorğusunun scope-unu ASP.NET Core açır. `AddDbContext` ilə qeyd edilən DbContext standart olaraq həmin scope daxilində paylaşılır. Bu qeydiyyat qaydasının adı scoped-dir; onu az sonra ayrıca nümunə ilə yoxlayacağıq.

Bir HTTP sorğusu və onun scope-u

```
Scope A acilir.
    DbContext elde edilir.
    SQL sorqusu 1: sifarisleri oxu.
    SQL sorqusu 2: mehsullari oxu.
    Cavab hazirlanir.
Scope A baglanir.
```

SQL baza ilə işləmək üçün sorğu dilidir. Bir HTTP sorğusunda bir neçə SQL sorğusu ola bilər. Hər SQL sorğusu üçün ayrıca HTTP scope-u açılmır.

Console proqramında, yəni terminalda işləyən tətbiqdə, HTTP sorğusunun avtomatik sərhədi yoxdur. Orada işi təşkil edən kod ayrıca scope aça bilər. Fon işində də sərhədi müəyyən etmək lazımdır. **Fon işi** istifadəçinin hazırkı HTTP sorğusundan ayrı emal olunan işdir.

Buradakı nəticə: **scope bir işin idarəetmə sərhədidir; onu kimin açması tətbiqin necə işləməsindən asılıdır.**

Resolve container-dən xidmət əldə etməkdir

İndi adı qoya bilərik: **resolve** container-dən qeydiyyata uyğun xidmət istəmək və onu əldə etməkdir. Container lazım gələrsə obyekti və onun asılılıqlarını yaradır, qayda icazə verirsə mövcud obyekti qaytarır.

C# - artıq açılmış scope üçün fraqment

```
var printer =
    scope.ServiceProvider.GetRequiredService<Printer>();
```

`scope` bizim açdığımız iş sərhədidir. `scope.ServiceProvider` həmin scope daxilində xidmət təmin edən mexanizmdir. **IServiceProvider** bu təminetmə imkanını ifadə edən .NET müqaviləsidir.

`GetRequiredService<Printer>()` Printer istədiyimizi bildirir. `<Printer>` burada tələb olunan tipi göstərir. Printer qeydiyyatda yoxdursa GetRequiredService xəta verir. `GetService<Printer>()` isə qeydiyyat tapılmayanda boş nəticə verə bilər.

Üç əməliyyatı qarışdırma:

- `new Printer()` obyekti birbaşa yaradır.
- `GetRequiredService<Printer>()` onu container-dən istəyir. Yeni və ya mövcud obyektin verilməsini qeydiyyat müəyyən edir.
- `printer.Print()` artıq alınmış obyektin metodunu çağırır. Bu ayrıca resolve deyil.

OrderService constructor-da Printer istəyəndə də container həmin asılılığı təmin edir. Buna görə adi biznes sinfinin hər metodunda container axtarmağa ehtiyac yoxdur. Nümunələrdə açıq GetRequiredService çağırışı prosesi görmək üçündür.

Qeydiyyat siyahısı provider və lifetime ayrı şeylərdir

ServiceCollection xidmətlərin necə təmin ediləcəyinə dair qeydiyyat siyahısıdır. **BuildServiceProvider** bu siyahıdan xidmət təmin edən container qurur. **CreateScope** onun daxilində ayrıca iş sərhədi açır.

Lifetime müəyyən qeydiyyat üçün yeni obyekt yaratma və mövcud obyekti paylaşma qaydasıdır. Üç əsas alternativ var:

C# - bunlardan yalnız birini seçirik

```
services.AddTransient<Printer>();
services.AddScoped<Printer>();
services.AddSingleton<Printer>();
```

Transient hər ayrıca resolve üçün yeni nüsxə, scoped eyni scope üçün bir nüsxə, singleton həmin container və qeydiyyat üçün bir nüsxə paylaşır. Sonrakı səhifələrdə bunları ayrıca izləyəcəyik.

C# - hazırlıq fraqmenti

```
var services = new ServiceCollection();
services.AddScoped<Printer>();

using var provider = services.BuildServiceProvider();
using var scope = provider.CreateScope();

var printer =
    scope.ServiceProvider.GetRequiredService<Printer>();
```

`AddScoped` burada Printer yaratmır; qaydanı qeyd edir. Provider qurulması da bu nümunədə Printer yaratmır. Printer ilk dəfə tələb olunanda yaradılır.

`using var` dəyişənin aid olduğu kod sahəsindən çıxanda onun Dispose təmizləməsini çağırmağı təşkil edir. Bu nümunədə scope provider-dən sonra yaradıldığı üçün çıxışda əvvəl scope, sonra provider bağlanır. Dispose-un mənasını ayrıca təmizlənmə hissəsində geniş açacağıq.

Transient eyni scope-da da ayrı obyektlər verir

C# - qeydiyyat

```
services.AddTransient<Printer>();
```

Tutaq ki, iki ayrı scope açılıb. Hərəsindən iki dəfə Printer istəyirik:

C# - dörd ayrıca resolve

```
var a = scope1.ServiceProvider.GetRequiredService<Printer>();
var b = scope1.ServiceProvider.GetRequiredService<Printer>();
var c = scope2.ServiceProvider.GetRequiredService<Printer>();
var d = scope2.ServiceProvider.GetRequiredService<Printer>();
```

Hər nömrə ayrı obyektdir

```
Scope 1: a -> obyekt 1
         b -> obyekt 2

Scope 2: c -> obyekt 3
         d -> obyekt 4

Cemi: 4 Printer obyekti.
```

Transient qaydası eyni scope daxilində də əvvəlki nüsxəni paylaşmır. Dörd ayrıca resolve dörd nüsxə verdi.

Amma `a.Print(); a.Print();` yazsaq, artıq aldığımız eyni a obyektində iki metod çağırışı edərik. **Transient hər metod çağırışında yenilənmək deyil.**

Transient ayrıca alınan xidmətin müstəqil nüsxəsi lazım olanda uyğun ola bilər. Seçimi yalnız “yüngül sinifdir” fikri ilə etmə; obyektin vəziyyətini kimlərin paylaşmalı olduğunu da nəzərə al.

Scoped eyni scope daxilində eyni obyekti verir

C# - qeydiyyat

```
services.AddScoped<Printer>();
```

Əvvəlki səhifədəki eyni dörd resolve çağırışı indi başqa nəticə verir:

İcra ardıcıllığı

```
Scope 1:
    a istenir -> obyekt 1 yaradilir.
    b istenir -> obyekt 1 qaytarilir.

Scope 2:
    c istenir -> obyekt 2 yaradilir.
    d istenir -> obyekt 2 qaytarilir.

Cemi: 2 Printer obyekti.
```

Birinci scope-da a ilə b eyni obyektə istinad edir. İkinci scope-da c ilə d eyni obyektə istinad edir. a ilə c isə ayrıdır.

Adi HTTP tətbiqində sifariş və ödəniş xidmətləri eyni sorğunun scope-undan eyni scoped Printer qeydiyyatını istəsə, eyni printer-i ala bilər. Növbəti sorğunun scope-u ayrıca printer alar.

Scoped “bir istifadəçiyə bir obyekt” demək deyil. Eyni istifadəçinin iki ayrı sorğusu adətən iki ayrı scope-dur. Scoped “bir thread-ə bir obyekt” də demək deyil. Thread kod icra edən axındır; scope-un sərhədi həmin axının adı ilə müəyyən edilmir.

DbContext-in `AddDbContext` ilə standart scoped qeydiyyatı bir iş daxilində onun paylaşılmasına kömək edir. Eyni DbContext-i eyni anda paralel baza əməliyyatlarında istifadə etmək isə uyğun deyil; scoped olmaq onu avtomatik təhlükəsiz etmir.

Singleton müxtəlif scope-larda da eyni obyekti verir

C# - qeydiyyat

```
services.AddSingleton<Printer>();
```

Eyni dörd resolve çağırışı bu dəfə yalnız bir Printer obyekti ilə işləyir:

İcra ardıcıllığı

```
Scope 1:
    a istenir -> obyekt 1 yaradilir.
    b istenir -> obyekt 1 qaytarilir.

Scope 2:
    c istenir -> obyekt 1 qaytarilir.
    d istenir -> obyekt 1 qaytarilir.

Cemi: 1 Printer obyekti.
```

Burada singleton ilk dəfə istənəndə yaranır. Sonrakı scope-lar da həmin container və qeydiyyat üçün eyni nüsxəni alır. Başqa container və ya başqa server prosesində ayrıca singleton ola bilər. **Proses** əməliyyat sistemində işləyən proqram nüsxəsidir.

Singleton sorğular arasında paylaşılır. Buna görə içində “hazırkı istifadəçinin sifarişi” kimi dəyişən məlumat saxlamaq istifadəçilərin məlumatını qarışdıra bilər.

Thread-safe eyni anda istifadə zamanı düzgün davranışı qorumaq deməkdir. Container-in özü xidmət təmin etməyi təhlükəsiz təşkil edə bilər, amma yaratdığı sinfin daxilindəki dəyişiklikləri avtomatik təhlükəsiz etmir. Singleton seçimi paylaşılma ehtiyacına və paralel istifadəyə görə əsaslandırılmalıdır.

Bir nüsxə yaratmaq həmişə daha düzgün dizayn demək deyil. Hansı məlumatın kimlərlə paylaşılacağını əvvəl müəyyən et.

Eyni obyekt olmasını ReferenceEquals ilə yoxlayaq

`ReferenceEquals(a, b)` a və b-nin eyni obyektə istinad edib-etmədiyini yoxlayır. **True** doğru, **False** yanlış nəticədir. Printer daxilindəki mətnlərin bərabər olması obyektlərin eyni olması demək deyil.

C# - dörd resolve-dan sonra yoxlama

```
Console.WriteLine(ReferenceEquals(a, b));
Console.WriteLine(ReferenceEquals(c, d));
Console.WriteLine(ReferenceEquals(a, c));
```

Lifetime	a ilə b	c ilə d	a ilə c
Transient	False	False	False
Scoped	True	True	False
Singleton	True	True	True

Birinci iki sütun eyni scope daxilini yoxlayır. Son sütun isə ayrı scope-ların aldığı obyektləri müqayisə edir.

Hər hərf ayrı obyekt deməkdir

```
Transient: Scope 1 -> A, B    Scope 2 -> C, D
Scoped:    Scope 1 -> A, A    Scope 2 -> B, B
Singleton: Scope 1 -> A, A    Scope 2 -> A, A
```

Bu cədvəl eyni container-də eyni qeydiyyatın dörd dəfə ayrıca istənməsi üçündür. Müxtəlif qeydiyyatların eyni obyektə bağlanması ayrıca qərardır. Ona görə “eyni sinif həmişə eyni singleton-dur” kimi ümumi hökm vermirik.

Console təcrübəsini işə salmaq üçün hazırlıq

.NET SDK C# layihəsini qurmaq və işlətmək üçün alətlər toplusudur. `dotnet` həmin alətlərin terminaldan çağırıldığı əmrdir. Bu dərsin nümunələri .NET 10 ilə yoxlanır.

Yeni Console layihəsi yaratmaq və DI kitabxanasını əlavə etmək üçün:

Terminal əmrləri

```
dotnet new console -n DiDemo
cd DiDemo
dotnet add package Microsoft.Extensions.DependencyInjection
```

Paket - package layihəyə əlavə edilən hazır kitabxanadır. Buradakı paket ServiceCollection və digər DI imkanlarını təqdim edir. `using Microsoft.Extensions.DependencyInjection;` isə həmin adları kodda qısa istifadə etməyə imkan verir.

Növbəti səhifədəki tam kodu layihənin **Program.cs** faylına yaz. Sonra işlət:

Terminal əmri

```
dotnet run
```

Əvvəl AddTransient ilə işlədirik. Sonra yalnız həmin qeydiyyat sətrini AddScoped ilə əvəz edib yenidən işlədirik. Üçüncü dəfə AddSingleton ilə yoxlayırıq. Üç qeydiyyatı birlikdə əlavə etmirik.

Kodda scope-ları əllə açmağımızın səbəbi Console proqramında avtomatik HTTP sorğusu olmamasıdır. Bu iki sərhəd lifetime fərqini ayrıca müşahidə etməyə imkan verir.

Console üçün tam işləyən lifetime proqramı

Bu variantda Printer-in boş sinif olması kifayətdir. Məqsəd qəbzi deyil, obyektlərin eyni və ya ayrı olmasını yoxlamaqdır.

C# - Program.cs

```
using System;
using Microsoft.Extensions.DependencyInjection;

var services = new ServiceCollection();
services.AddTransient<Printer>();

using var provider = services.BuildServiceProvider();
using var scope1 = provider.CreateScope();
using var scope2 = provider.CreateScope();

var a = scope1.ServiceProvider.GetRequiredService<Printer>();
var b = scope1.ServiceProvider.GetRequiredService<Printer>();
var c = scope2.ServiceProvider.GetRequiredService<Printer>();
var d = scope2.ServiceProvider.GetRequiredService<Printer>();

Console.WriteLine("AB: " + ReferenceEquals(a, b));
Console.WriteLine("CD: " + ReferenceEquals(c, d));
Console.WriteLine("AC: " + ReferenceEquals(a, c));

public class Printer
{
}
```

`AB` birinci scope daxilində a ilə b-ni, `CD` ikinci scope daxilində c ilə d-ni, `AC` isə ayrı scope-lardan a ilə c-ni müqayisə edir. + burada mətnləri birləşdirir.

`using var` sətirləri iki scope-un və əsas provider-in proqramın işi bitəndə bağlanmasını təşkil edir. Eyni anda açıq iki scope olması onların bir-birinə qarışması demək deyil.

Proqramın üç ayrı işə salınmasının nəticəsi

Tam proqramda qeydiyyat sətrini dəyişərək alınan nəticələr:

AddTransient<Printer>() ilə

```
AB: False
CD: False
AC: False
```

Dörd resolve, dörd obyekt. Scope-lar eyni olsa da hər ayrıca resolve yeni nüsxə alır.

AddScoped<Printer>() ilə

```
AB: True
CD: True
AC: False
```

Dörd resolve, iki obyekt. Hər scope bir nüsxəni daxilində paylaşır.

AddSingleton<Printer>() ilə

```
AB: True
CD: True
AC: True
```

Dörd resolve, bir obyekt. Scope-lar arasında da həmin nüsxə paylaşılır.

Buradakı nəticələr dörd ayrıca GetRequiredService çağırışı üçün nəzərdə tutulub. Əgər `var b = a;` yazsan, b sadəcə a-dakı istinadı alar. Hətta transient qeydiyyatda da bu zaman ReferenceEquals(a, b) True olar, çünki ikinci resolve etməmisən.

İki dəyişən və iki metod çağırışı yeni resolve deyil

İstinadı ötürməklə yeni obyekt yaratmağın fərqini ayrıca yoxlayaq. Burada adi qəbz hazırlayan Printer tərifindən istifadə olunur:

C# - artıq açılmış scope daxilində

```
var a = scope.ServiceProvider.GetRequiredService<Printer>();
var b = a;

Console.WriteLine(ReferenceEquals(a, b));
Console.WriteLine(a.Print());
Console.WriteLine(a.Print());
```

Birinci nəticə **True** olur. Sonrakı iki sətir eyni a obyektinin Print metodunu çağırır. Container-dən yalnız bir dəfə xidmət istəmişik.

Bu iki sətir isə iki resolve-dur

```
var a = scope.ServiceProvider.GetRequiredService<Printer>();
var b = scope.ServiceProvider.GetRequiredService<Printer>();
```

Bu halda eyni və ya ayrı obyektin verilməsini lifetime müəyyən edir.

Eyni fərq constructor injection üçün də keçərlidir. Bir OrderService obyektinə printer verilib sahədə saxlanıbsa, onun Complete metodunu təkrar çağırmaq həmin sahəni avtomatik başqa printer-lə əvəz etmir.

Obyektin mövcud olması ilə obyekt üzərində əməliyyat çağırılması ayrı hadisələrdir. Yeni resolve görmədən “burada transient olduğu üçün dəyişəcək” nəticəsinə gəlmə.

Interface və OrderService-i container ilə birləşdirək

Əvvəldə IPrinter, Printer, PdfPrinter və OrderService-i əllə birləşdirdik. İndi həmin siniflər ayrı fayllarda layihədə qalsın. Aşağıdakı Program.cs onların container vasitəsilə qurulmasını göstərir:

C# - Program.cs, əvvəlki interface və siniflərlə

```
using Microsoft.Extensions.DependencyInjection;

var services = new ServiceCollection();
services.AddScoped<IPrinter, Printer>();
services.AddScoped<OrderService>();

using var provider = services.BuildServiceProvider();
using var scope = provider.CreateScope();

var orders = scope.ServiceProvider
    .GetRequiredService<OrderService>();

Console.WriteLine(orders.Complete());
```

Nəticə **Adi qebz** olur. Birinci qeydiyyatda soldakı IPrinter tələb olunan müqavilə, sağdakı Printer isə konkret reallaşdırmadır.

OrderService istənəndə container onun IPrinter asılılığını görür. Bu scope üçün Printer yaradır və onu OrderService-in konstruktoruna verir. Eyni scope-da yenidən OrderService istənəndə scoped qaydasına görə həmin OrderService qaytarılır.

`AddScoped<IPrinter, PdfPrinter>()` seçsən, eyni sifariş kodunun nəticəsi **PDF qebzi** olar. Alternativ qeydiyyat əvvəlkini əvəz edir; bu təcrübədə iki sətri birlikdə yazmırıq.

Burada GetRequiredService yalnız proqramın birləşdirmə yerindədir. OrderService-in daxilində container istifadə edilmədi; sinif ehtiyacını yenə constructor-da bildirir.

Real HTTP sorğusunda scope-u framework açır

Yeni veb layihəsi üçün `dotnet new web -n DiWebDemo` yaz, həmin qovluğa keç və Program.cs-ni aşağıdakı kodla əvəz et. ASP.NET Core-un veb layihəsində DI imkanları hazır təqdim olunur.

C# - veb layihəsinin tam Program.cs faylı

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddScoped<Printer>();

var app = builder.Build();

app.MapGet("/printers", (Printer a, Printer b) =>
{
    return new
    {
        A = a.Id,
        B = b.Id,
        Same = ReferenceEquals(a, b)
    };
});

app.Run();

public class Printer
{
    public Guid Id { get; } = Guid.NewGuid();
}
```

`MapGet` həmin ünvana gələn GET sorğusunu qarşılayacaq kodu müəyyən edir. GET məlumat almaq üsuludur. `=>` burada çağırılacaq kod blokunu göstərir. ASP.NET Core qeydiyyatda olan Printer parametrlərini sorğunun scope-undan təmin edir.

`Id` obyekti izləmək üçün verdiyimiz tanıdıcıdır. `Guid.NewGuid()` yeni tanıdıcı dəyər yaradır. `get` onun oxuna bildiyini göstərir. `new { ... }` A, B və Same məlumatları olan sadə nəticə obyekti qurur.

İki HTTP sorğusunun nəticəsini oxuyaq

Veb layihəsinin qovluğunda aşağıdakı əmri işlət:

Terminal əmri

```
dotnet run --urls http://localhost:5087
```

Brauzerdə `http://localhost:5087/printers` ünvanını aç. `localhost` öz kompüterini, `5087` isə proqramın dinlədiyi portu bildirir. Port şəbəkə əlaqəsinin həmin kompüterdə hansı girişə çatacağını seçir.

Cavabda A və B tanıdıcıları, bir də Same nəticəsi görünəcək. Bu, **JSON**, yəni məlumatı ad və dəyər cütləri ilə yazan mətn formatıdır. **Same** eyni obyekt olub-olmadığını bildirir.

Scoped variantda birinci cavabın A və B dəyərləri eyni olur. Səhifəni yeniləyib ikinci HTTP sorğusunu göndərəndə yenə öz aralarında eyni, amma əvvəlki sorğudakından fərqli dəyərlər alınır.

Qeydiyyat	Bir cavabda A və B	İkinci sorğunun obyekti
Transient	Ayrı, Same false	Yeni obyektlər
Scoped	Eyni, Same true	Əvvəlki sorğudan ayrı
Singleton	Eyni, Same true	Əvvəlki sorğu ilə eyni

Bu üç variant ayrıca işə salınır. Tətbiqi bağlayıb yenidən açmaq yeni container yaradır; singleton da əvvəlki prosesdəki obyektin davamı olmur.

Kodda CreateScope yazmadıq. **HTTP scope-unu ASP.NET Core özü idarə etdi.** Printer parametrlərinin avtomatik verilməsi də endpoint inteqrasiyasıdır. Endpoint müəyyən ünvan və sorğu üsuluna cavab verən giriş nöqtəsidir.

Constructor method və property injection fərqi

Constructor injection-da asılılıq obyekt yaranarkən verilir. İlk nümunələrimizdə bunu gördük. Obyektin daimi ehtiyacını constructor-da göstərmək onun nəyə bağlı olduğunu açıq edir.

Method injection asılılığın konkret əməliyyata verilməsidir. Aşağıdakı sinif əvvəlki IPrinter müqaviləsindən istifadə edir:

C# - method injection

```
public class ReceiptJob
{
    public string Run(IPrinter printer)
    {
        return printer.Print();
    }
}
```

Çağıran kod `new ReceiptJob().Run(new Printer())` yaza bilər. Printer yalnız həmin çağırışa verilir.

Property injection asılılığın obyektin xüsusiyyətinə sonradan verilməsidir. Property oxuma və ya yazma imkanı verən xüsusiyyətdir:

C# - property injection

```
public class ReceiptJob
{
    public IPrinter Printer { get; set; } = null!;
}
```

Çağıran kod obyekt yaradandan sonra `job.Printer = new Printer();` yazır. `set` dəyər yazmağa imkan verir. `null!` yalnız boş dəyərlə bağlı kompilyator xəbərdarlığını susdurur; obyekt yaratmır və təhlükəsizlik təmin etmir.

Standart .NET container adi sinfin property-lərini avtomatik doldurmur, onun hər metoduna da özbaşına parametr vermir. Veb endpoint-ləri kimi xüsusi yerlərdəki dəstək framework-ün ayrıca inteqrasiyasıdır.

Singleton scoped asılılığı saxlayanda problem yaranır

Tutaq ki, printer-i scoped, hesabat xidmətini singleton qeyd etdik. Hesabat xidməti printer-i constructor-la alıb öz sahəsində saxlayır:

C# - lifetime uyğunsuzluğu olan qeydiyyat

```
services.AddScoped<Printer>();
services.AddSingleton<ReportRunner>();
```

C# - ReportRunner.cs

```
public class ReportRunner
{
    private Printer _printer;

    public ReportRunner(Printer printer)
    {
        _printer = printer;
    }
}
```

Bir iş sərhədi üçün nəzərdə tutulan Printer indi uzun yaşayan ReportRunner tərəfindən tutulur. Buna **captive dependency**, yəni qısa sərhədli asılılığın uzun yaşayan obyekt tərəfindən saxlanması deyilir.

Scope yoxlaması aktivdirsə container belə asılılığı rədd edə bilər. Console təcrübəsində `BuildServiceProvider(new ServiceProviderOptions { ValidateScopes = true })` bu yoxlamanı aktivləşdirir. `true` burada seçimin açıq olduğunu bildirir.

Hesabat sorğuya aiddirsə onu da scoped etmək mümkün həldir. Həqiqətən uzun yaşayan fon xidmətidirsə, hər iş üçün ayrıca scope yaradıb printer-i həmin scope daxilində istifadə etmək olar. Növbəti səhifədə bunu göstərəcəyik.

Uzun yaşayan xidmət hər iş üçün ayrıca scope aça bilər

IServiceScopeFactory yeni scope yaratmaq üçün istifadə olunan .NET xidmətidir. Burada ReportRunner bir işə başlayanda scope açır, printer-i alır və iş bitəndə scope-u bağlayır. Printer-in Print metodu əvvəlki kimi mətn qaytarır.

C# - ReportRunner.cs, düzgün alternativ

```
public class ReportRunner
{
    private IServiceScopeFactory _scopeFactory;

    public ReportRunner(IServiceScopeFactory scopeFactory)
    {
        _scopeFactory = scopeFactory;
    }

    public string Run()
    {
        using var scope = _scopeFactory.CreateScope();
        var printer = scope.ServiceProvider
            .GetRequiredService<Printer>();

        return printer.Print();
    }
}
```

Bu sinif üçün `using Microsoft.Extensions.DependencyInjection;` lazımdır. Qeydiyyatlar yenə Printer üçün scoped, ReportRunner üçün singleton ola bilər.

İki ayrı Run çağırışı iki ayrı scope açır. Hər çağırış öz scoped printer-ini alır. Printer singleton-un sahəsində saxlanılmır və scope-dan kənarda istifadə üçün qaytarılmır; nəticə olan mətn qaytarılır.

Asinxron təmizlənmə tələb edən resurslarla uyğun `CreateAsyncScope` və `await using` forması istifadə edilə bilər. Asinxron əməliyyat tamamlanmanı gözləyərkən icra axınını bloklamamağa imkan verir. Bunun təfərrüatı ayrıca async dərsindədir.

Singleton transient obyekti saxlayanda o da dəyişmir

Əvvəlki ReportRunner-in printer-i sahədə saxlayan constructor variantına qayıdaq. Bu dəfə qeydiyyat belə olsun:

C# - bu iki qeydiyyat ayrıca ssenaridir

```
services.AddTransient<Printer>();
services.AddSingleton<ReportRunner>();
```

ReportRunner ilk dəfə yarananda container onun üçün bir Printer yaradır. ReportRunner həmin istinadı sahədə saxlayır. Sonrakı metod çağırışları eyni ReportRunner və eyni printer-lə işləyir.

Nə baş verir

```
ReportRunner ilk defe istenir.
    Printer A yaradilir ve ona verilir.

ReportRunner yeniden istenir.
    Movcud ReportRunner qaytarilir.
    Onun sahesinde hele Printer A var.
```

Printer-in transient qeydiyyatı mövcud sahəni avtomatik yeniləmir. Yeni printer almaq üçün ayrıca resolve və ya uyğun yaradılma mexanizmi lazımdır.

Singleton-un transient asılılıq alması hər halda avtomatik xəta deyil. Amma asılılığın faktiki istifadə müddəti uzanır və eyni vaxtda istifadəyə uyğunluğu da düşünülməlidir. **Qeydiyyat adı ilə obyektin real istifadə müddətini ayrı yoxla.**

Singleton-un hər iş üçün yeni scope açdığı əvvəlki həll printer-in paylaşılma sərhədini açıq idarə edirdi. Seçim obyektin hansı işə aid olmasına görə verilir.

Dispose resursu buraxır GC yaddaşı geri alır

Fayl və baza bağlantısı kimi resurslar iş bitəndə buraxılmalıdır. **Dispose** bu təmizləmə əməliyyatının adıdır. **IDisposable** onu təqdim edən müqavilədir. `using` çıxışda Dispose çağırılmasını təşkil edir.

Container-in yaratdığı və təmizlənməsini idarə etdiyi, IDisposable olan xidmətlər üçün:

Xidmət	Təmizlənmə sərhədi
Scope-dan alınmış transient	Həmin scope bağlananda
Scoped	Həmin scope bağlananda
Singleton	Əsas provider bağlananda

Hər xidmətin Dispose metodu olmur. Sadə boş Printer sinfimizdə belə resurs yoxdur. Təmizlənmə qaydası resurs idarə edən xidmətlərdə əhəmiyyətlidir.

Container-in verdiyi ortaq asılılığı onu istifadə edən hər sinif ayrıca bağlamamalıdır. Öz yaratdığın scope-u isə bağlamalısan. Root, yəni əsas provider-dən alınmış disposable transient-lər provider bağlanana qədər saxlanıla bilər; belə xidmətlər üçün uyğun scope istifadə et.

GC - Garbage Collector .NET-in artıq əlçatmaz olan idarə olunan obyektlərin yaddaşını sonradan geri alan mexanizmidir. İdarə olunan dedikdə yaddaşa .NET-in nəzarəti nəzərdə tutulur. Dispose çağırılması obyektin dərhal yaddaşdan silinməsi deyil.

Hazır obyekt özün yaradılıb `AddSingleton(hazirObyekt)` ilə verilirsə, onun təmizlənmə məsuliyyəti də səndə qalır. **IAsyncDisposable** təmizlənməsi asinxron gözlənə bilən resursların müqaviləsidir; burada uyğun await using işlədilir.

Bir müqavilə üçün bir neçə həll qeyd ediləndə

Əvvəlki IPrinter, Printer və PdfPrinter təriflərini istifadə edək:

C# - iki ayrı reallaşdırmanın qeydiyyatı

```
services.AddTransient<IPrinter, Printer>();
services.AddTransient<IPrinter, PdfPrinter>();
```

Standart .NET container-də tək IPrinter istənəndə bu sadə qeydiyyatların sonuncusu, yəni PdfPrinter seçilir. İkisini birlikdə almaq üçün **IEnumerable<IPrinter>** istənilə bilər. Bu, IPrinter obyektləri ardıcıllığını ifadə edən müqavilədir.

C# - açılmış scope daxilində

```
var printers = scope.ServiceProvider
    .GetRequiredService<IEnumerable<IPrinter>>();

foreach (var printer in printers)
{
    Console.WriteLine(printer.Print());
}
```

`foreach` ardıcıllığın hər elementi üçün daxildəki kodu işləyir. Nəticədə əvvəl **Adi qebz**, sonra **PDF qebzi** yazılır. Eyni xidmət üçün təsadüfən bir neçə qeydiyyat əlavə etmək gözlədiyin seçimi dəyişə bilər.

Factory - yaradıcı funksiya obyektin necə yaradılacağını kodla bildirə bilər. Məsələn, ayrıca alternativ qeydiyyat `services.AddTransient<Printer>(provider => new Printer());` yazıla bilər. `provider` funksiya parametridir; burada istifadə edilmir. Container yeni nüsxə tələb olunanda bu funksiyanı çağırır.

Factory qeydiyyatı constructor asılılıqlarını qarışıq şəkildə biznes metodlarının içində axtarmağa ehtiyac yaratmır. Yaradılma qərarı yenə tətbiqin qurulduğu sərhəddə saxlanılır.

DI IoC və DIP eyni sualı cavablandırmır

Bu üç anlayışı artıq gördüyümüz kodla bağlayaq:

Anlayış	Bizim nümunədə mənası
DI - Dependency Injection	Printer obyektini OrderService-ə kənardan veririk.
IoC - Inversion of Control	Yaradılma və əlaqələndirmənin idarəsini container təşkil edir.
DIP - Dependency Inversion Principle	Sifarişin əsas məntiqini konkret printer detalına deyil IPrinter müqaviləsinə bağlayırıq.

IoC idarəetmənin müəyyən hissəsinin başqa mexanizmə keçməsi barədə daha geniş anlayışdır. DI onun asılılıqları verməyə aid üsuludur.

DIP SOLID prinsiplərinin D hərfidir. SOLID dəyişiklikləri idarə etməyə kömək edən beş dizayn prinsipinin qısa adıdır. DIP-də əsas iş qaydası ilə texniki detal sabit müqaviləyə bağlanır. Burada sifariş qaydası əsas iş, printer-in konkret quruluşu texniki detaldır.

Abstraksiya lazımi davranışı detallardan ayıran təsvir və ya müqavilədir. Bizim nümunədə IPrinter bu rolu oynayır. Interface parametri yazıb metodun içində yenidən konkret Printer yaratmaq verilmiş asılılığın faydasını itirə bilər.

DI avtomatik olaraq bütün dizayn problemlərini həll etmir. Hər sadə dəyər və ya kiçik sinif üçün ayrıca interface yaratmaq məcburi deyil. Dəyişən texniki asılılıqları və paylaşılma ehtiyaclarını müəyyən etmək daha vacibdir.

Tez rast gəlinən problemləri səbəbi ilə tanı

Qeydiyyat çatışmır. Constructor IPrinter istəyir, amma container onun necə təmin olunacağını bilmir. Uyğun qeydiyyatı əlavə et. Sinif faylının mövcud olması təkbaşına xidmət qeydiyyatı deyil.

Dairəvi asılılıq. A yaratmaq üçün B, B yaratmaq üçün A lazımdır. Buna circular dependency deyilir. Container məsuliyyətləri sənin yerinə bölmür. Hansı kodun hansı işi görməli olduğunu yenidən düşün.

Gizli asılılıq. Sinif constructor-da ehtiyaclarını göstərmək əvəzinə hər metodda provider-dən xidmət axtarır. Buna Service Locator yanaşması deyilir. Ehtiyaclar gizlənir və sinfi əllə qurub yoxlamaq çətinləşir. Adi biznes sinfində constructor injection daha açıqdır.

Sorğudan uzun yaşayan iş. HTTP sorğusunda alınmış DbContext-i nəzarətsiz fon işinə verirsən, sorğu isə bitir. Scope bağlananda iş artıq təmizlənmiş resursla qarşılaşa bilər. Fon işinin öz scope-u və idarə olunan tamamlanması olmalıdır.

Bir neçə əsas container. Qeydiyyat zamanı lazımsız BuildServiceProvider çağırışları ayrıca provider-lər yaradıb fərqli singleton nüsxələri və təmizlənmə sərhədləri yarada bilər. ASP.NET Core tətbiqində əsas container-i adətən builder.Build qurur.

Çoxlu constructor asılılığı. Bir sinif on fərqli xidmət tələb edirsə, onun üzərinə çox iş yığılmış ola bilər. Təkcə constructor-u qısaltmaq yox, məsuliyyətləri aydın bölmək barədə düşün.

Özünü nümunə üzərində yoxla

1. Constructor-a Printer yazdım. Obyekt avtomatik yarandı? Xeyr. Əllə istifadədə onu çağıran kod yaradır və verir. Container istifadəsində qeydiyyata uyğun təmin edilir.

2. DI etmək üçün mütləq interface lazımdır? Xeyr. Konkret Printer-i kənardan vermək də DI-dır. Interface dəyişə bilən həlli müqavilə arxasında saxlamağa kömək edir.

3. AddScoped sətrində Printer yaranır? Bu tip qeydiyyat nümunəsində xeyr. Qayda yazılır; nüsxə xidmət tələb olunanda yaranır.

4. İki HTTP sorğusu neçə scope-dur? Adi ASP.NET Core sorğu emalında iki ayrı scope. Eyni istifadəçinin göndərməsi bunu dəyişmir.

5. Scoped printer-i eyni scope-dan iki dəfə aldım. Eyni obyekt alınır. Ayrı scope-dan aldıqda ayrıca obyekt alınır.

6. Transient printer-i bir dəfə alıb Print-i beş dəfə çağırdım. Bir obyekt üzərində beş metod çağırışı etdin. Beş resolve etmədin.

7. Singleton hər serverdə eyni obyekt olur? Xeyr. Hər prosesin və container-in öz qeydiyyat sərhədi var.

8. Scope-u EF Core açır? Adi HTTP scope-unu ASP.NET Core açır. DbContext həmin sərhəddə istifadə edilən xidmət ola bilər.

9. Eyni DbContext eyni scope-da olduğuna görə paralel istifadə təhlükəsizdir? Xeyr. Scoped olmaq paralel istifadəyə avtomatik uyğunluq vermir.

10. Mənə “DI-ni izah et” deyilsə: sinif ehtiyac duyduğu obyekti kənardan alır; container bunu qeydiyyata görə təşkil edə bilər; scope iş sərhədidir; resolve xidməti əldə etməkdir; lifetime həmin xidmətdə nüsxələrin paylaşılma qaydasıdır.

Mənbələr və nümunələri necə oxumalı

Əsas tədris ardıcıllığı və qəbz nümunələri bu dərs üçün yazılıb. .NET davranışlarını yoxlamaq üçün aşağıdakı rəsmi sənədlərdən istifadə edilir. İlk oxunuşda əlavə mənbələri açmaq məcburi deyil.

- Microsoft Learn - DI və container
- Microsoft Learn - transient scoped singleton
- Microsoft Learn - təmizlənmə və lifetime səhvləri
- Microsoft Learn - ASP.NET Core HTTP sorğusunda DI
- Microsoft Learn - DbContext ömrü və paralel istifadə

Tam Program.cs yazılan nümunə həmin layihənin faylını bütövlükdə əvəz edir. **Fraqment** isə artıq hazırlanmış layihənin göstərilən hissəsinə əlavə olunan kod parçasıdır. Ayrıca sinif tərifləri layihədə ayrı fayllarda saxlanıla bilər.

Eyni adlı Printer və OrderService tərifləri alternativ öyrənmə mərhələləridir. Hamısını bir layihəyə üst-üstə əlavə etmək eyni adın təkrar tərifi xətasına səbəb olar. Ekran nəticələri həmin səhifədə göstərilən alternativə aiddir.

Praktik məşq: Console nümunəsində tək qeydiyyat sətrini dəyiş, nəticəni əvvəl özün proqnozlaşdır, sonra işə sal. Daha sonra veb nümunəsində iki ayrı HTTP sorğusunu müqayisə et. Burada məqsəd adları əzbərləmək deyil, hər yeni obyektin hansı addımda yarandığını göstərə bilməkdir.